PROBLEMARIO ERRORES DE PROGRAMACION

Emanuel Islas Quintos

Cristian Avila Rodríguez

Contenido

Análisis de errores 3
1.- Error de Redondeo Binario 3
2. Pérdida de Precisión por Magnitud(I**EEE 754**) 3
3.-Comparación Directa con == 6
4. Acumulación de Errores en Bucles 7
5.-Cancelación por Resta (Loss of Significance) 9
6.-Desbordamiento Silencioso (Overflow) 10
7.-Conversión Estrecha (Narrowing Primitive Conversion) 13
Problemas investigados: 15
1. OS Command Injection (Inyección de Comandos en Sistema Operativo) 15
2. SQL Injection (Inyección SQL) 15
3. Heap-Based Buffer Overflow en SolidWorks eDrawings (CVE-2024-10204) 15
4. Integer Overflow en Blender al Cargar Imágenes (CVE-2022-0545) 16
5. Memory Corruption por Manipulación de Memoria en C/C++ 16
6. Acceso Directo a Objetos Inseguro (IDOR) 16
7. Comparación de NaN siempre falla 16
8. Orden incorrecto al sumar números flotantes 17
Problemas de error absoluto y relativo 18
Ejemplo 1: Error absoluto 18
Ejemplo 2: Error relativo 18
Ejemplo 3: Medición de altura de una persona 19
Ejemplo 4: Contando monedas 20
Ejemplo 5: Calificaciones de un examen 21
Ejemplo 6: Medición del peso de una persona 22
Cronograma de actividades 23
Fuentes de información 23

Análisis de errores

1.- Error de Redondeo Binario

El error más común es intentar representar números decimales exactos (como 0.1) en un sistema binario. Esto causa que operaciones simples devuelvan resultados inesperados.

- **Ejemplo:** System.out.println(0.1 + 0.2); imprime 0.30000000000000004 en lugar de 0.3.
- **Causa:** Fracciones decimales no tienen una representación binaria finita.

```
public class Main
{
    public static void main(String[] args)
    {
        System.out.println(0.1 + 0.2);
    }
}

SALIDA
0.3000000000000004

Esto para por que los números decimales no se pueden representar
exactamente en binario
```

2. Pérdida de Precisión por Magnitud(IEEE 754)

Cuando se operan números con magnitudes muy diferentes, el número más pequeño puede "desaparecer" debido a que no cabe en la mantisa del número más grande.

- **Ejemplo:** Sumar 1.0 a un double extremadamente grande puede no cambiar el valor original.

```
public class PerdidaPrecision {
   public static void main(String[] args) {
      // Un double tiene aproximadamente 15-17 dígitos significativos de precisión
      double numeroGrande = 1.0e16; // 1 seguido de 16 ceros
      double numeroPequeno = 1.0;

      double resultado = numeroGrande + numeroPequeno;

      System.out.println("--- Demostración de Pérdida de Precisión ---");
      System.out.println("Número Grande:   " + numeroGrande);
      System.out.println("Número Pequeño:  " + numeroPequeno);
```

```
        System.out.println("Suma Resultante: " + resultado);

        // Verificación lógica
        if (resultado == numeroGrande) {
            System.out.println("\nRESULTADO: El número pequeño 'desapareció'.");
            System.out.println("La suma es igual al número original debido a la falta de bits en la
mantisa.");
        }

        // --- SOLUCIÓN USANDO BIGDECIMAL ---
        java.math.BigDecimal bdGrande = new java.math.BigDecimal("1.0e16");
        java.math.BigDecimal bdPequeno = new java.math.BigDecimal("1.0");
        java.math.BigDecimal bdResultado = bdGrande.add(bdPequeno);

        System.out.println("\n--- Solución con BigDecimal ---");
        System.out.println("Suma Exacta:     " + bdResultado.toPlainString());
    }
}
```

```java
public class Main
 {
     public static void main(String[] args)
     {
         // Un double tiene aproximadamente 15-17 dígitos significativos
de precisión
         double numeroGrande = 1.0e16; // 1 seguido de 16 ceros
         double numeroPequeno = 1.0;

         double resultado = numeroGrande + numeroPequeno;

         System.out.println("--- Demostración de Pérdida de Precisión --
-");
         System.out.println("Número Grande:   " + numeroGrande);
         System.out.println("Número Pequeño:  " + numeroPequeno);
         System.out.println("Suma Resultante: " + resultado);

     }
 }
```

SALIDA

```
--- Demostración de Pérdida de Precisión ---

Número Grande:   1.0E16

Número Pequeño:  1.0

Suma Resultante: 1.0E16

1.0E16 + 1 no existe como número representable
```

3.-Comparación Directa con ==

Debido a los errores de redondeo mencionados, comparar dos números double usando == suele fallar.

- **Solución:** Se debe usar un margen de error (épsilon).

  java

  ```
  if (Math.abs(a - b) < 0.00001) { /* son iguales */ }
  ```

```
public class ComparacionDouble {
   public static void main(String[] args) {
      // 1. El Problema: Error de redondeo en aritmética de punto flotante
      double a = 0.1 + 0.1 + 0.1;
      double b = 0.3;

      System.out.println("Valor de a (0.1 * 3): " + a);
      System.out.println("Valor de b:          " + b);
```

```
class Main
{
    public static void main(String[] args)
    {
        double a = 0.1 + 0.1 + 0.1;
        double b = 0.3;

        System.out.println("Valor de a (0.1 * 3): " + a);
        System.out.println("Valor de b:           " + b);

    }
}

SALIDA
0.3000000000000004

Esto para por que los números decimales no se pueden representar
exactamente en binario
```

```
      // 2. Intento de comparación directa (FALLARÁ)
      System.out.println("\n--- Comparación con '==' ---");
      if (a == b) {
         System.out.println("Resultado: Son iguales");
      } else {
         System.out.println("Resultado: SON DIFERENTES (Error esperado)");
```

```
        }

        // 3. La Solución: Uso de un margen de error (Épsilon)
        double epsilon = 0.00001; // Definimos la tolerancia

        System.out.println("\n--- Comparación con Épsilon (" + epsilon + ") ---");
        if (Math.abs(a - b) < epsilon) {
            System.out.println("Resultado: Son iguales (dentro del margen de error)");
        } else {
            System.out.println("Resultado: Son diferentes");
        }
    }
}
```

4. Acumulación de Errores en Bucles

Realizar miles de operaciones aritméticas consecutivas con double acumula pequeños errores de redondeo que pueden resultar en una desviación significativa al final del proceso.

```
import java.math.BigDecimal;

public class AcumulacionErroresBucle {
    public static void main(String[] args) {
        int iteraciones = 1000000; // Un millón de sumas
        double incremento = 0.1;

        // 1. Acumulación usando 'double'
        double sumaDouble = 0.0;
        for (int i = 0; i < iteraciones; i++) {
            sumaDouble += incremento;
        }

        // El resultado esperado debería ser exactamente 100,000.0
        double esperado = iteraciones * incremento;

        System.out.println("--- Acumulación en Bucle (1,000,000 de iteraciones) ---");
        System.out.println("Resultado esperado: " + esperado);
        System.out.println("Resultado double:   " + sumaDouble);
        System.out.println("Diferencia (Error): " + (sumaDouble - esperado));
```

```java
public void acumulacionErroresBucles() {
        int iteraciones = 1000000; // Un millón de sumas
        double incremento = 0.1;

        // 1. Acumulación usando 'double'
        double sumaDouble = 0.0;
        for (int i = 0; i < iteraciones; i++) {
            sumaDouble += incremento;
        }

        // El resultado esperado debería ser exactamente 100,000.0
        double esperado = iteraciones * incremento;

        System.out.println("--- Acumulación en Bucle (1,000,000 de
iteraciones) ---");
        System.out.println("Resultado esperado: " + esperado);
        System.out.println("Resultado double:   " + sumaDouble);
        System.out.println("Diferencia (Error): " + (sumaDouble -
esperado));

    }
```

SALIDA:

Resultado esperado: 100000.0

Resultado double: 100000.00000133288

Diferencia (Error): 1.3328826753422618E-6

Esto para porque las pequeñas operaciones guardan pequeños errores en memoria que al hacer la misma operación sumando el resultado se genera un error notable.

```java
// 2. Solución usando 'BigDecimal' para precisión absoluta
BigDecimal sumaBD = BigDecimal.ZERO;
BigDecimal incrementoBD = new BigDecimal("0.1");

for (int i = 0; i < iteraciones; i++) {
    sumaBD = sumaBD.add(incrementoBD);
}
```

```
        System.out.println("\n--- Solución con BigDecimal ---");
        System.out.println("Resultado exacto:   " + sumaBD.toPlainString());

        // 3. Verificación de error
        if (sumaDouble != esperado) {
            System.out.println("\nNOTA: El error de 'double' es notable tras un millón de sumas.");
        }
    }
}
```

5.-Cancelación por Resta (Loss of Significance)

Ocurre cuando restas dos números muy cercanos entre sí. La mayoría de los dígitos significativos se cancelan, dejando solo el error de redondeo como el "resultado" aparente.

java

```
public class CancelacionPorResta {
    public static void main(String[] args) {
        // Dos números muy grandes y muy cercanos
        double x = 1234567890.1234567;
        double y = 1234567890.1234560;

        // El resultado esperado es 0.0000007
        double resultado = x - y;

        System.out.println("Resultado real: " + resultado);
        // En 2026 notarás que el resultado es 0.0 o un valor basura
        // debido a que los últimos dígitos se perdieron al almacenar x e y.
    }
}
```

```java
public void cancelacionResta() {
        // Dos números muy grandes y muy cercanos
        double x = 1234567890.1234567;
        double y = 1234567890.1234560;

        // El resultado esperado es 0.0000007
        double resultado = x - y;

        System.out.println("Resultado real: " + resultado);
        // En 2026 notarás que el resultado es 0.0 o un valor basura
        // debido a que los últimos dígitos se perdieron al almacenar
x e y.

    }
```

```
SALIDA: Cancelación por Resta:

Resultado real: 7.152557373046875E-7

Esto sucede por las cifras significativas y un error de redondeo
```

6.-Desbordamiento Silencioso (Overflow)

A diferencia de los errores de decimales, este ocurre con tipos enteros (int, long). Si superas el valor máximo, Java no lanza una excepción; simplemente "da la vuelta" al número más pequeño (negativo), lo que destruye la exactitud del cálculo.

java

```java
public class DesbordamientoEntero {
    public static void main(String[] args) {
        int max = Integer.MAX_VALUE; // 2,147,483,647
        int resultado = max + 1;

        System.out.println("Máximo int: " + max);
        System.out.println("Máximo + 1: " + resultado); // Imprime -2147483648

        // Solución: Usar Math.addExact para lanzar una excepción si ocurre
```

```
        try {
            Math.addExact(max, 1);
        } catch (ArithmeticException e) {
            System.out.println("Error detectado: ¡Desbordamiento!");
        }
    }
}
```

```java
public void desbordamientoSilencioso() {
        int max = Integer.MAX_VALUE; // 2,147,483,647
        int resultado = max + 1;

        System.out.println("Máximo int: " + max);
        System.out.println("Máximo + 1: " + resultado); // Imprime -
2147483648

        // Solución: Usar Math.addExact para lanzar una excepción si
ocurre
        try {
            Math.addExact(max, 1);
        } catch (ArithmeticException e) {
            System.out.println("Error detectado: ¡Desbordamiento!");
        }

    }
```

SALIDA:

6. Desbordamiento Silencioso:

Máximo int: 2147483647

Máximo + 1: -2147483648

Error detectado: ¡Desbordamiento!

7.-Conversión Estrecha (Narrowing Primitive Conversion)

Al convertir un tipo de mayor capacidad a uno menor (de double a int o de long a int), Java simplemente trunca los bits sobrantes, lo que puede cambiar drásticamente el valor original sin previo aviso.

Java

```java
public class ErrorConversion {
    public static void main(String[] args) {
        double valorDouble = 3.14159e10; // Un número muy grande
        int valorInt = (int) valorDouble; // Casting explícito

        System.out.println("Valor Original: " + valorDouble);
        System.out.println("Valor Truncado: " + valorInt);
        // Imprime 2147483647 (el máximo int) porque el double no cabe.
    }
}
```

```java
public void conversionEstrecha() {
        double valorDouble = 3.14159e10; // Un número muy grande
        int valorInt = (int) valorDouble; // Casting explícito

        System.out.println("Valor Original: " + valorDouble);
        System.out.println("Valor Truncado: " + valorInt);
        // Imprime 2147483647 (el máximo int) porque el double no
cabe.


    }
```

SALIDA:

7. Conversión Estrecha:

Valor Original: 3.14159E10

Valor Truncado: 2147483647

Sucede al convertir un número muy grande guardado en un variable de

Problemas investigados:

1. OS Command Injection (Inyección de Comandos en Sistema Operativo)

Software: Aplicaciones de consola / servidores / scripts que construyen comandos dinámicos
Descripción: Si los datos de usuario se incrustan en comandos de sistema sin sanitización, un atacante puede ejecutar comandos arbitrarios.
Cómo reproducir: En un script que llama system("ls " + user_input) pasando ; rm -rf / como input provoca ejecución no deseada.
Fuente: Tipos comunes de vulnerabilidades CWE.

(Hellas, 2023)

2. SQL Injection (Inyección SQL)

Software: Bases de datos conectadas desde aplicaciones (PHP, .NET, Java, VBA en Excel con ADODB)
Descripción: Un atacante puede modificar consultas SQL inyectando código malicioso si no hay parámetros.
Cómo reproducir: En un formulario de búsqueda sin parámetros ejecutar ' OR '1'='1 para devolver todos los registros.
Fuente: Vulnerabilidades comunes en sistemas con BD.

(Pruitt, 2026)

3. Heap-Based Buffer Overflow en SolidWorks eDrawings (CVE-2024-10204)

Software: SolidWorks / eDrawings
Descripción: Error en lectura de archivos X_B o SAT puede causar desbordamiento en el montón, permitiendo ejecución de código arbitrario.
Cómo reproducir: Abrir un archivo X_B/SAT especialmente manipulado en eDrawings de SolidWorks 2024/2025.
Fuente: CVE-2024-10204 (NVD).

(*NVD - CVE-2024-10204*, n.d.)

4. Integer Overflow en Blender al Cargar Imágenes (CVE-2022-0545)

Software: Blender 2.80-3.0.1 y variantes

Descripción: El overflow en la lectura de imágenes puede provocar corrupción de memoria o permitir ejecución de código al procesar imágenes maliciosas.

Cómo reproducir: Cargar una imagen específicamente manipulada en Blender 2.x / 3.0.1.

Fuente: CVE list para Blender.

(*Blender CVEs and Security Vulnerabilities - OpenCVE*, n.d.)

5. Memory Corruption por Manipulación de Memoria en C/C++

Software: Programas escritos en C/C++

Descripción: Acceso fuera de límites de un arreglo o memoria no inicializada puede corromper datos o provocar crashes.

Cómo reproducir: Leer o escribir a array[-1] o usar valores no inicializados en cálculos.

Fuente: Tipos clásicos de errores de software.

(Wikipedia contributors, 2025)

6. Acceso Directo a Objetos Inseguro (IDOR)

Software: Aplicaciones web con backend de BD

Descripción: Acceso a objetos de otros usuarios porque los parámetros no se verifican contra permisos.

Cómo reproducir: Cambiar el ID de recurso en la URL y acceder a información ajena.

Fuente: Vulnerabilidades frecuentes en aplicaciones con BD.

(*Vulnerabilidades Comunes En Empresas Y Cómo Evitarlas*, n.d.)

7. Comparación de NaN siempre falla

Software / Lenguaje: C, C++, Java, Python, R, MATLAB

Nombre del problema: *NaN comparison anomaly*

Descripción

Según el estándar **IEEE 754**, cualquier comparación con NaN **siempre devuelve falso**, incluso NaN == NaN.

Cómo reproducir

double x = Math.sqrt(-1);

System.out.println(x == x); // false

8. Orden incorrecto al sumar números flotantes

Software / Lenguaje: C, Java, Python, R
Nombre del problema: *Non-associativity of floating-point addition*

Descripción

La suma en punto flotante **no es asociativa**:

$$(a + b) + c \neq a + (b + c)$$

Cómo reproducir

```
double a = 1e16;
    double b = -1e16;
    double c = 1.0;

    System.out.println((a + b) + c); // 1.0
    System.out.println(a + (b + c)); // 0.0
```

Problemas de error absoluto y relativo

Ejemplo 1: Error absoluto

1.- Contexto:

Se mide la longitud de una mesa con una regla. El valor real de la mesa es conocido gracias a un calibrador preciso.

2.- Datos:

- Valor real: $V_{\text{real}} = 2.00$ m
- Valor medido: $V_{\text{aprox}} = 1.95$ m

3.- Fórmula:

$$E_a = | V_{\text{real}} - V_{\text{aprox}} |$$

4.- Sustitución:

$$E_a = | 2.00 - 1.95 |$$

5.- Resultado:

$$E_a = 0.05 \text{ m}$$

Interpretación:

El error absoluto indica que la medición se desvió 0.05 m del valor real.

Ejemplo 2: Error relativo

1.- Contexto:

Usando la misma medición de la mesa, queremos saber qué tan grande es el error comparado con el valor real.

2.- Datos:

- Valor real: $V_{\text{real}} = 2.00$ m
- Valor aproximado: $V_{\text{aprox}} = 1.95$ m
- Error absoluto: $E_a = 0.05$ m(del ejemplo anterior)

3.- Fórmula:

$$E_r = \frac{E_a}{|\ V_{\text{real}}\ |}$$
$$E_r\% = E_r \times 100$$

4.- Sustitución:

$$E_r = \frac{0.05}{2.00} = 0.025$$
$$E_r\% = 0.025 \times 100 = 2.5\%$$

5.- Resultado:

$$E_r = 0.025 \text{o} E_r\% = 2.5\%$$

Interpretación:
El error relativo indica que la medición difiere 2.5% del valor real, mostrando la magnitud del error en términos relativos.

Ejemplo 3: Medición de altura de una persona

1.- Contexto:
Se mide la altura de una persona con una cinta métrica. Queremos calcular el error absoluto y relativo de la medición comparando con la altura real conocida (por ejemplo, medida con un tallímetro profesional).

2.- Datos:

- Valor real: $V_{\text{real}} = 1.75$ m
- Valor medido: $V_{\text{aprox}} = 1.72$ m

3.- Fórmula:

- Error absoluto:

$$E_a = |\ V_{\text{real}} - V_{\text{aprox}}\ |$$

- Error relativo:

$$E_r = \frac{E_a}{|\ V_{\text{real}}\ |} \times 100$$

4.- Sustitución:

$$E_a = | 1.75 - 1.72 | = 0.03 \text{ m}$$
$$E_r = \frac{0.03}{1.75} \times 100 \approx 1.71\%$$

5.- Resultado:

- **Error absoluto:** 0.03 m
- **Error relativo:** 1.71%

Interpretación:
La medición se desvió **3 cm del valor real**, lo que representa **aproximadamente un 1.7% del valor real**. Esto indica que la medición es bastante precisa, pero no exacta.

Ejemplo 4: Contando monedas

1.- Contexto:
Se quiere contar el número de monedas en una alcancía, pero alguien hizo una estimación rápida en lugar de contarlas exactamente. Queremos calcular el error absoluto y relativo de la estimación.

2.- Datos:

- Número real de monedas: $V_{\text{real}} = 120$
- Número estimado: $V_{\text{aprox}} = 115$

3.- Fórmula:

- Error absoluto:

$$E_a = | V_{\text{real}} - V_{\text{aprox}} |$$

- Error relativo:

$$E_r = \frac{E_a}{| V_{\text{real}} |} \times 100$$

4.- Sustitución:

$$E_a = | 120 - 115 | = 5$$
$$E_r = \frac{5}{120} \times 100 \approx 4.17\%$$

5.- Resultado:

- **Error absoluto:** 5 monedas
- **Error relativo:** 4.17 %

Interpretación:
La estimación se equivocó en 5 monedas, lo que representa aproximadamente un **4% del total**, mostrando que la estimación es razonable, pero no exacta.

Ejemplo 5: Calificaciones de un examen

1.- Contexto:
Un profesor revisa un examen y comete un pequeño error al registrar la calificación de un estudiante. Queremos calcular el error absoluto y relativo.

2.- Datos:

- Calificación real: $V_{\text{real}} = 95$
- Calificación registrada: $V_{\text{aprox}} = 90$

3.- Fórmula:

- Error absoluto:

$$E_a = | V_{\text{real}} - V_{\text{aprox}} |$$

- Error relativo:

$$E_r = \frac{E_a}{| V_{\text{real}} |} \times 100$$

4.- Sustitución:

$$E_a = | 95 - 90 | = 5$$
$$E_r = \frac{5}{95} \times 100 \approx 5.26\%$$

5.- Resultado:

- **Error absoluto:** 5 puntos
- **Error relativo:** 5.26 %

Interpretación:
El error absoluto indica que la calificación fue registrada 5 puntos por debajo del valor real. El error relativo nos muestra que esto representa **aproximadamente el 5% del puntaje total**, un error moderado que podría afectar el promedio del estudiante.

Ejemplo 6: Medición del peso de una persona

1.- Contexto:
Una persona se pesa en una báscula casera que no es muy precisa. Queremos calcular el error absoluto y relativo comparando con su peso real medido en una báscula profesional.

2.- Datos:

- Peso real: $V_{\text{real}} = 70$ kg
- Peso medido: $V_{\text{aprox}} = 68.5$ kg

3.- Fórmula:

- Error absoluto:

$$E_a = | V_{\text{real}} - V_{\text{aprox}} |$$

- Error relativo:

$$E_r = \frac{E_a}{| V_{\text{real}} |} \times 100$$

4.- Sustitución:

$$E_a = | 70 - 68.5 | = 1.5 \text{ kg}$$
$$E_r = \frac{1.5}{70} \times 100 \approx 2.14\%$$

5.- Resultado:

- **Error absoluto:** 1.5 kg
- **Error relativo:** 2.14 %

Interpretación:
El peso medido se desvió **1.5 kg del valor real**, lo que representa aproximadamente un **2% del peso real**. Esto indica que la báscula casera es bastante cercana, pero no totalmente exacta.

Cronograma de actividades

Encargado	Actividad
Emanuel Islas Quintos	Errores De programación Documento
Emanuel Islas Quintos	Investigar mas errores de programacion
Cristian Avila Rodriguez	Problemas de error relativo y absoluto
Cristian Avial Rodriguez	Organización del documento

Fuentes de información

Blender CVEs and security vulnerabilities - OpenCVE. (n.d.). *https://app.opencve.io/cve/?product=blender&vendor=blender&utm_source=chatgpt.com*

NVD - CVE-2024-10204. (n.d.). *https://nvd.nist.gov/vuln/detail/CVE-2024-10204?utm_source=chatgpt.com*

Pruitt, M. (2026, January 7). Software security vulnerabilities. Kiuwan. *https://www.kiuwan.com/blog/types-software-vulnerabilities/?utm_source=chatgpt.com*

Hellas, M. S. C. F. R. &. T. –. (2023, January 13). Simple coding mistakes that can lead to critical vulnerabilities - IoTAC. IoTAC. *https://iotac.eu/simple-coding-mistakes-that-can-lead-to-critical-vulnerabilities/?utm_source=chatgpt.com*

Vulnerabilidades Comunes en Empresas y Cómo Evitarlas. (n.d.). https://blog.tecnetone.com/vulnerabilidades-comunes-en-empresas-y-c%C3%B3mo-evitarlas?utm_source=chatgpt.com

Wikipedia contributors. (2025, September 18). *Common weakness enumeration*. https://en.wikipedia.org/wiki/Common_Weakness_Enumeration?utm_source=chatgpt.com